CE152: Object-Oriented Programming

Encapsulation

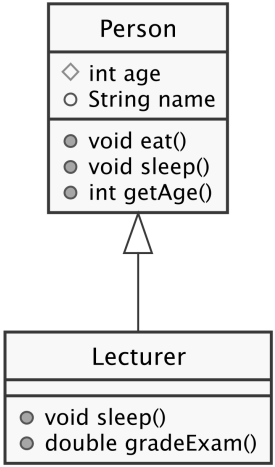
Dr. Sebastian Halder

CSEE

2023

Last video

Inheritance: `class Lecturer extends Person`



Encapsulation

Why do we worry about Encapsulation?

- *Encapsulation* is not about security (in the sense of encryption) but about control
- What we want to control is how variables in one class can be accessed by other classes
- We want to restrict access to certain variables (information hiding)
- It is *good practice* to use the must restrictive access level possible for any given variable or method

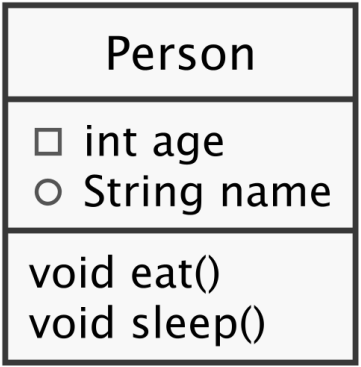
Coupling

- The reason is a concept called *coupling* (the dependence of one piece of code on another) which we want to minimise
- If there are many ways for class `A` to interact with class `B` there is a high chance that `A` will have to change if `B` changes
- Where the changes need to be made will also be more difficult to identify
- If there is just one way because we restrict the interaction as much as possible we can easily identify necessary changes or modify `B` in a way so that no changes in `A` are necessary at all!
- This is of course also possible without *encapsulation* (some languages like Python do not have access modifiers and support OOP)

Access modifiers in Java

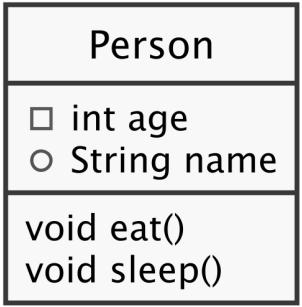
- `public`: the member (variable or method) can be accessed from any class
- `private`: the member can only be accessed from the owning class
- `protected`: the member can only be accessed from sub-classes
- `default`: the member can only be accessed from classes in the same package

Encapsulation



- In this UML diagram a little square appeared next to age and a circle next to name
- The square indicates age is *private*
- The circle indicates name is *public*
- Other similar keywords are *protected* (diamond; access only by subclasses) and *package private* (default in Java; triangle; access only from same package)

Let's make the age private

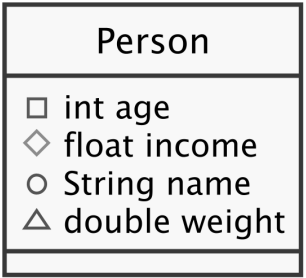


- The age cannot be changed so we set it to private
- For now we are ok with anyone changing the name of the Person so we set it to public

```
class Person {
    private int age;
    public String name;

    Person(String name){
      this.name = name;
      this.age = 0;
    }
}
```

Other access modifiers



- Does everyone remember the difference between `int`, `float` and `double`?
- Can we make the whole class private? Protected?

```
private int age;
protected float income;
public String name;
double weight;
```

Live coding

- Demonstrate effect of access modifiers on members

Summary

Summary

- Access modifiers (`public`, `protected`, `private` and default) can be used to restrict how a class can be interacted with
- This is important to minimise *coupling* (the dependency of one class on another)
- Minimal *coupling* leads to easier maintenance of code

Thank you and see you in the next video!